INFORME CONFIDENCIAL

Capacitacion IA Neural Labs

Sprint de adopcion completado. Diagnostico inicial, resultados obtenidos, impacto por miembro del equipo y recomendaciones para escalar.

PREPARADO POR	EMPRESA	CTO	STACK
Alan Buscaglia	Neural Labs	Damian	Angular / .NET / Azure DevOps

01 — CONTEXTO

La empresa antes de empezar

Neural Labs es una empresa orientada al cliente donde la prioridad es la entrega. Si el cliente necesita algo, se hace, incluso bugfixing sobre la marcha. El problema estructural mas grande al momento de iniciar la capacitacion era el tiempo de desarrollo: todo se retrasaba.

Las releases eran semestrales, los conflictos de versiones entre productos y subproductos eran constantes, y el context switching de los desarrolladores agravaba cada demora.

> *Stack principal: Angular, .NET, Azure DevOps. QA manual sin testing automatizado. Documentacion dispersa entre SharePoint y documentos internos sin un estandar claro.*

Procesos de desarrollo

- Releases grandes y poco frecuentes (una por semestre), lo que generaba acumulacion de cambios, conflictos de versiones y una desorganizacion considerable.
- No existia testing unitario ni e2e. No habia un lugar donde los tests estuvieran escritos. Platform tenia un documento de funcionalidades, pero no tests ejecutables.
- La revision de PRs se hacia con Anthropic pero con un prompt muy ligero, sin estructura ni criterio estandarizado.
- El flujo de trabajo dependia de reuniones semanales para definir tareas y parar la maquina para verificar que todo funcionara. Mucho del proceso era manual y oral.

Documentacion

Server: cobertura 3/10. Documentos tipo "loop" que se perdian.

Platform: cobertura 6/10. Se documentaban las partes nuevas y se actualizaba, pero el volumen de informacion hacia imposible tener todo en la cabeza. Se perdia el motivo que originaba cada cambio.

No existian reglas documentadas ni convenciones de codigo formalizadas.

Uso de IA antes de la capacitacion

Cada miembro del equipo usaba IA de forma aislada y con resultados dispares. No habia un criterio unificado ni un flujo compartido.

J

Jordi

QA — 8 anos de experiencia — Testing manual funcional exploratorio

La usaba para ahorrar tiempo en casos donde no tenia margen, le preguntaba a la IA que haria en su lugar. Le agilizaba la creacion de manuales.

M

Miguel Angel

Server + Platform — 14 anos — Distribucion de tareas + analisis

La usaba continuamente pero de forma limitada. Creia que habia que ser analista para exprimirla al 100%. Estimaba que el 80% del codigo lo hacia la IA. Hacia mil cosas a la vez, mucho se podia automatizar.

D

Diego

Orchestrator / Director — Soporte tecnico

Su principal frustracion era que la IA olvidaba contexto. Habia que repetirle todo constantemente.

Uso de IA antes de la capacitacion

R

Raul

Server + Angular — 2 anos

Sentia que la IA no respetaba su forma de programar, no generaba codigo como el lo haria. Reconocia que cada vez era mas util pero con desconfianza.

Q

Joaquin

Platform — Angular + .NET — 2 semanas al iniciar

Venia con mente fresca, pro IA, buenos conceptos. Sin los vicios ni frustraciones del equipo. Buena fuente de ideas nuevas al no estar ligado a los problemas actuales.

F

Francisco

Server

La usaba para buscar errores y analizar problemas. Su posicion era clara: lo mas importante es como preguntarle y revisar lo que devuelve. No confiaba mucho en el desarrollo generado por IA.

Que se logro durante la capacitacion

Procesos y herramientas implementadas

- Se crearon skills a nivel repositorio: para iniciar repos, creacion de tareas y flujo de las mismas, creacion de release notes, y migracion/optimizacion de codigo existente a versiones mas actualizadas.
- La IA se integro como mediador de contexto, ayudando a extraer informacion util de forma eficaz. Ejemplo concreto: panel de tareas pendientes asistido por IA.
- Se empezo a usar la IA para creacion y split de tareas, algo que antes era completamente manual y dependia de reuniones.
- Se utiliza la IA para busqueda de alternativas y mejor camino a un resultado, no solo para generar codigo.
- La revision de codigo y PRs ahora tiene mas estructura que el prompt ligero original.
- Mejoria concreta en la comunicacion entre equipo y definicion de flujos.
- El equipo reporta ser mas rapido y ordenado a la hora de entregar y documentar.

Que se logro durante la capacitacion

QA

Jordi esta mucho mas integrado con la IA. Los documentos de QA ahora son asistidos por IA basandose en el code base y alimentados con documentos base. Playwright queda en evaluacion para implementar testing e2e.

Web

No se llego a implementar unit testing durante la capacitacion, pero el plan esta definido: primero actualizar y refactorear el codigo, despues meter tests. El equipo tiene claro el orden.

Management

Se descubrio que pasarle un PDF a la IA y pedirle que asista en el analisis, optimizando flujos tomando los use cases, ayuda considerablemente. Esto aplica directamente a su proceso de cotizacion y definicion de tareas.

Un concepto pendiente: IA no es solo un LLM

El equipo adopto agentes de IA (Claude Code, Codex, etc.) y eso esta muy bien, pero hay un salto conceptual que todavia falta dar. IA no es solo pedirle cosas a un modelo de lenguaje. Hay una capa entera de procesos inteligentes algoritmicos que pueden optimizar flujos sin necesidad de un LLM: automatizacion de CI/CD, analisis estatico, deteccion de patrones en codigo, pipelines de validacion, bots de triaje. Distinguir cuando usar un agente conversacional y cuando usar un proceso algoritmico determinista es la diferencia entre usar IA como herramienta y entender IA como disciplina.

> *El equipo esta motivado, con ganas de aprender y probar cosas nuevas. Hay mentalidad de mejora continua y voluntad real de optimizar procesos y comunicacion. Esto es probablemente el resultado mas valioso: no se impuso una herramienta, se genero un cambio de mentalidad.*

Evolucion por miembro del equipo

M

Miguel Angel

Server + Platform — 14 anos

ANTES

Usaba IA de forma limitada. Creia que habia que ser analista para sacarle provecho.

AHORA

Definio sus modelos preferidos segun complejidad: `codex / sonnet 4.5` para cosas simples, `opus 4.6` para cosas complejas. Entiende que no todo se resuelve con el mismo modelo y que la seleccion importa. Paso de "la uso pero no se si la exprimo" a tener criterio propio.

F

Francisco

Server

ANTES

Desconfianza en el desarrollo generado por IA. La usaba principalmente para buscar errores.

AHORA

Usa `codex 5.3` para programacion. Mantiene su postura critica (que es sana) pero la canalizo hacia un uso productivo en lugar de evitar la herramienta. Su enfoque de "como preguntarle y revisar lo que devuelve" es exactamente el correcto.

R

Raul

Server + Angular — 2 anos

ANTES

Frustracion porque la IA no programaba como el queria.

AHORA

Usa `opus 4.6` para programacion y documentacion, con `codex 5.3` como alternativa. Sigue teniendo desconfianza, lo cual es un punto a trabajar. Necesita optimizar sus procesos para que la herramienta se adapte a su forma de trabajar, no al reves.

Evolucion por miembro del equipo

D

Diego

Orchestrator / Director

ANTES

Su queja principal era que la IA olvidaba todo.

AHORA

Usa `codex 5.3` para programacion y `opus 4.6` para documentacion. Su workflow evoluciono al modelo ideal: piensa como lo quiere, pregunta, analiza, actua. Es el ejemplo de human-in-the-loop bien hecho. El problema del olvido se mitiga con el uso correcto de contexto y memoria persistente.

Q

Joaquin

Platform — 2 semanas al iniciar

ANTES

Dos semanas en la empresa, mente fresca, pro IA.

AHORA

Usa `codex 5.3` para programacion y manejo de tareas con skills. Al no tener los vicios del equipo, adopto las herramientas de forma natural. Es un buen referente para mostrar que el approach funciona cuando no hay resistencia previa.

J

Jordi

QA — 8 anos

ANTES

Usaba IA para ahorrar tiempo y crear manuales.

AHORA

Documentos de QA asistidos por IA basados en code base, evaluando Playwright para e2e. Su rol se transformo de "QA manual que a veces usa IA" a "QA que tiene la IA integrada en su flujo de documentacion y analisis".

05 — COMPARATIVA

Puntos de dolor: antes vs ahora

PUNTO DE DOLOR ORIGINAL	ESTADO ACTUAL	ESTADO
Todo se retrasaba, tiempos de desarrollo largos	Tiempos mejorados. Mas rapido y ordenado para entregar y documentar.	Mejorado
Releases semestrales con conflictos de versiones	Se trabaja en release notes automatizadas, pero el ciclo de releases aun no cambio estructuralmente.	Parcial
Sin testing unitario ni e2e	Web: plan definido (refactor primero, tests despues). QA: evaluando Playwright.	Pendiente
Documentacion dispersa y sin estandar	Mejoria en documentacion asistida por IA. Skills de repo creados. Falta formalizar el estandar completo.	Parcial
IA olvidaba contexto	El equipo ya usa Engram (memoria persistente), pero no de manera correcta. Falta estructurar como se guardan decisiones y convenciones para que el contexto realmente persista entre sesiones.	Parcial
IA no respeta el estilo del dev	Parcialmente resuelto con seleccion de modelos. Se resuelve completamente con skills custom.	Parcial
PRs con revision muy ligera	Mejorado, con IA integrada en revision de codigo y PRs.	Mejorado
No hay reglas documentadas de codigo	Se crearon skills de repo, pero falta documentar reglas como especificacion formal.	Parcial

Problemas que persisten

- La IA a veces no hace lo que se le dice o "se olvida" del trabajo. El equipo ya tiene Engram instalado pero no lo esta usando de forma efectiva. Falta definir que se guarda, cuando y con que estructura. Combinado con SDD, esto se resuelve de raiz.
- Desconfianza de parte de algunos miembros (Raul, Francisco). Es sana cuando se canaliza como verificacion, pero puede frenar la adopcion si no se trabaja.
- El testing automatizado sigue sin implementarse. Es el gap mas grande y mas riesgoso para la calidad.

Proximos pasos para seguir escalando

01

Implementar SDD (Spec-Driven Development)

El flujo estructurado de exploracion, propuesta, especificacion, diseno, tareas, implementacion y verificacion resuelve directamente los problemas de "la IA no hace lo que le digo" y "se olvida". Si la IA trabaja contra una spec, no improvisa.

02

Usar Engram correctamente

El equipo ya tiene Engram instalado, lo cual es un buen primer paso, pero no lo esta aprovechando. La memoria persistente solo sirve si se define que guardar: decisiones arquitectonicas, convenciones de equipo, contexto de tareas en progreso, patrones de migracion. Sin esa disciplina, es como tener un disco rigido y no guardar nada en el.

03

Actualizar el stack de herramientas con Gentle AI

El equipo tiene las piezas sueltas (agentes, Engram, skills) pero no un ecosistema unificado. Gentle AI integra todo en un solo comando: SDD, memoria persistente, documentacion on-demand, skills custom y configuracion de agentes. Actualizar a Gentle AI les da un unico punto de gobierno sobre todas las herramientas de IA que ya usan, en lugar de tener cada pieza configurada por separado.

04

Priorizar testing automatizado

Definir un punto de ruptura claro. Desde este commit en adelante, todo codigo nuevo tiene tests. No intentar cubrir todo el legacy de golpe, eso paraliza. Unit tests primero, e2e con Playwright despues.

05

Trabajar la desconfianza con evidencia

Raul y Francisco necesitan ver resultados concretos en su propio codigo. No se les convence con charlas, se les convence cuando un skill custom genera codigo que respeta sus patrones y la verificacion les muestra que el resultado es correcto.

Proximos pasos para seguir escalando

06

Evaluar mejor los pasos de implementacion

Antes de asignar una tarea, usar la IA para hacer el split y la estimacion. Esto ya se empezo a hacer pero necesita formalizarse como parte del proceso, no como algo opcional.

07

Automatizar release notes

Ya se creo un skill para esto. El siguiente paso es integrarlo en el flujo de CI/CD para que no dependa de que alguien se acuerde de ejecutarlo.

08

Documentar el por que, no solo el que

Cada cambio deberia documentar no solo que se hizo sino por que. Engram cubre esto a nivel de IA, pero tambien necesitan un estandar humano (ADRs o similar) para que el equipo que venga despues entienda las decisiones.

09

Mejorar la especificacion dentro del codigo

Tienen que situar que hay y que deberia haber. Marcar un punto de ruptura y decir "desde aqui para adelante se hace de la forma esperada". Esto es exactamente lo que SDD formaliza.

10

Distinguir IA de LLM: incorporar procesos algoritmicos

El equipo aprendio a trabajar con agentes conversacionales y eso es un gran avance. Pero IA no es solo un LLM. Hay procesos inteligentes algoritmicos (automatizacion de CI/CD, analisis estatico, deteccion de patrones, pipelines de validacion, bots de triaje) que resuelven problemas de forma determinista sin necesidad de un modelo de lenguaje. Saber cuando usar un agente y cuando usar un proceso algoritmico es lo que separa a un equipo que usa IA de un equipo que entiende IA.

07 — CONCLUSION

El equipo tiene la motivacion. Ahora necesita el framework.

El equipo de Neural Labs paso de usar IA de forma aislada y con resultados inconsistentes a tener skills de repositorio, flujos asistidos, documentacion mejorada y una mentalidad de mejora continua. Los tiempos mejoraron y la comunicacion entre equipo tambien.

Lo que falta es estructurar lo que aprendieron. Unificar el stack con Gentle AI para tener un solo punto de gobierno. Usar Engram de forma efectiva, no solo tenerlo instalado. Implementar SDD para que el flujo sea deterministico. Incorporar testing para que la calidad no dependa solo de Jordi. Y dar el salto conceptual de entender que IA no es solo un LLM: hay procesos algoritmicos que resuelven problemas enteros sin necesidad de un modelo conversacional.

Para cualquier duda o seguimiento:

Alan Buscaglia

GDE & MVP — Founder, Gentleman Programming